

Autopilot Revenue Desk

An agent that turns weekend newsletter clicks and website visits into approved, personalised sales outreach and real CRM deals on graph8, with a human approving every send.

Team	Mahrukh Muzzamil, Ayesha Haroon, Malaika
Backend repo	github.com/AyeshaHaroon394/graph8-hackathon-backend
Frontend repo	github.com/MahrukhMuzzamil/graph8-hackathon-frontend
Stack	TypeScript · Node.js 20+ · Next.js 16 / React 19 · Tailwind 4
Platform	graph8 Developer API (be.graph8.com/api/v1)
Version	September 2026

Contents

1. The problem and the product
2. Architecture and infrastructure
3. Tech stack, data and storage
4. The 10-step pipeline: how each step works
5. Key logic in detail (scoring, finding people, writing emails, safety)
6. graph8 Work: Monday brief and approve-by-reply
7. Radar competitor detection
8. Learning from decisions
9. Weekend batch and demo weekend
10. Dashboard (Mission Control) and app tour
11. Modes and configuration
12. HTTP API reference
13. Postman: collection and sample requests
14. How to run and test every feature
15. graph8 endpoints used
16. Known limits and next steps

1. The problem and the product

Every week a software company's marketing team sends a newsletter and publishes articles. Over the weekend hundreds of people click. On Monday, sales receives a spreadsheet of those clicks, and nobody uses it: it's long, messy, and doesn't say who matters. Interested buyers are forgotten by the time anyone looks.

Autopilot Revenue Desk does that Monday work automatically. For every company that engaged, it:

1. collects the weekend's clicks and visits,
2. drops personal emails and students, then enriches and scores each company, with the reasons shown,
3. finds the real decision-makers (CTO, VP) in graph8's contact database,
4. saves the company and people in the graph8 CRM,
5. has graph8's AI write a personal email that mentions what their team read,
6. waits for a human to approve, reject or edit, in the dashboard or by replying in graph8 Work,
7. enrolls the approved contacts in a sequence (held on the live system),
8. reads the reply and drafts a response, prepares an AI call script, and
9. creates the deal in the CRM with a suggested next step.

It also posts a **Monday brief** to graph8 Work, checks **graph8 Radar** for competitors, and **learns** from every approve, reject and edit.

2. Architecture and infrastructure

Browser · Mission Control (Next.js)

→ REST + SSE →

Backend · Node.js orchestrator

→ HTTPS →

graph8 API · enrichment, search, CRM, sequences, AI, voice, Radar, Work

Part	What it is	Runs where
Frontend	Next.js 16 app, one page (<code>MissionControl1</code>) plus the tour. Talks only to the backend.	Port 3000 (<code>npm run dev</code> / <code>npm start</code>)
Backend	Plain <code>node:http</code> server (no framework) that runs the pipeline, streams progress, and proxies graph8.	Port 4000 (<code>npm start</code>)
graph8	All business data lives here: companies, contacts, lists, deals, tasks, sequences, Work messages, Radar.	be.graph8.com/api/v1
CI	GitHub Actions: TypeScript typecheck and a mock-mode smoke test on every push and pull request.	GitHub

Request flow

1. The user clicks **Run weekend**. The frontend calls `POST /api/batch`.
2. The backend finds the weekend's companies, starts one pipeline run per company in the background, and returns their ids at once (HTTP 202).

- Each run emits an event per step. The frontend listens with Server-Sent Events (`GET /api/events?runId=`) and polls `GET /api/runs` for the funnel.
- At step 6 the run pauses on an in-memory promise until a decision arrives from the dashboard (`POST /api/approvals/{id}`) or from a graph8 Work reply.

Reliability built into the graph8 client (`lib/graph8/client.ts`)

- Rate limit:** at most `G8_MAX_RPS` requests per second (default 4; graph8 allows 5), queued in order across all parallel runs.
- Retries:** 429, 502, 503, 504 (and 500 on reads) retried with exponential backoff, honouring `Retry-After` .
- Idempotency:** every write sends an `Idempotency-Key` derived from the run id (for example `run_ab12:deal`), so a retry never creates duplicates.
- Caching:** reads and searches cached in memory with a TTL (enrichment 1 h, Radar 10 min), which saves credits when many runs look up the same thing.
- Sandbox guard:** in sandbox mode nothing runs unless `GET /sandbox/status` confirms a sandbox.

3. Tech stack, data and storage

Layer	Choice	Why
Language	TypeScript (strict) end to end	One set of types shared by backend and frontend (<code>lib/pipeline/types.ts</code>)
Backend runtime	Node.js 20+, run with <code>tsx</code> ; built-ins only (<code>node:http</code> , <code>fetch</code> , <code>crypto</code> , <code>fs</code>)	No framework, no runtime dependencies: fast to start and hard to break
Frontend	Next.js 16 (App Router), React 19, Tailwind CSS 4	Fast UI work, dark mode, responsive
Realtime	Server-Sent Events	One-way progress stream; simpler than WebSockets
System of record	graph8 CRM	Companies, contacts, lists, deals, tasks and messages are stored in graph8, as the organisers asked
Local state	In-memory event log + JSON files in <code>backend/data/</code>	No separate database to run: the data that matters lives in graph8
CI	GitHub Actions	Typecheck + mock smoke test

What is stored where

Data	Store	Lifetime
Pipeline events (every step of every run)	In memory, last 2,000 events	Until restart or Reset
Pending approvals	In memory (promise per approval id, 30-minute timeout, then reject)	Until decided
Weekend company profiles (clicks, themes, run and deal ids)	<code>data/weekend-profiles.json</code> (atomic write via temp file + rename)	Persistent

Learning (approve / reject / edit history)	<code>data/learning.json</code>	Persistent; <code>POST /api/learning/reset</code> clears it
Companies, contacts, lists, deals, tasks	graph8 CRM	Persistent
Monday brief, approval requests, replies	graph8 Work channel <code>#autopilot-revenue-desk</code>	Persistent

`data/` and `.env` are git-ignored.

4. The 10-step pipeline

Each run is a `PipelineContext` passed through ten steps (`lib/pipeline/run.ts`). Every step emits `running` , then `done` , `skipped` (with a reason) or `failed` . Three kinds of stop keep one problem from breaking the demo:

- `StopPipeline` : a deliberate end (not a fit, rejected by the human). Later steps are marked skipped with the reason.
- `SkipStep` : one step can't run (for example no voice agent), but later steps still run.
- Any other error: the step is marked failed and the run stops.

#	Step	What it does	graph8 endpoints
1	Engagement	Collects the company's weekend signals (newsletter clicks, site visits, tracked links).	<code>GET /visitors/companies/events</code> , <code>GET /visitors/links/events</code> , newsletter subscriber timeline
2	Qualify	Enriches the company and scores fit and intent, with reasons; then applies learning and Radar.	<code>POST /enrichment/lookup/company</code> , <code>GET /enrichment/companies/gtm-profile</code> , <code>GET /intent/abm/companies</code> , <code>POST /hiring-signals/companies</code> , <code>GET /radar/competitors</code>
3	Committee	Finds the decision-makers at the company and ranks them by role.	<code>POST /search/contacts</code> (optional <code>POST /voice/research/org-chart</code>)
4	CRM	Creates a list, upserts the company and the contacts, then reads back contact ids.	<code>POST /lists</code> , <code>POST /companies</code> , <code>PUT /contacts/assert/batch</code> , <code>GET /contacts</code>
5	Outreach	graph8's AI writes an email to the top two people and a LinkedIn note to the champion.	<code>POST /sequencer/content/email/generate</code> , <code>POST /sequencer/content/linkedin/generate</code>
6	Approval	Pauses for a human: approve, reject or edit, in the dashboard or in graph8 Work.	<code>POST /work/messages</code> , <code>GET /work/conversations/{id}/messages</code>
7	Send	Sandbox: enrolls contacts in a sequence and reads the outbox. Live: held (dry run), or only your own test inbox.	<code>GET/POST /sequences</code> , <code>POST /sequences/{id}/contacts</code> , <code>GET /sandbox/outbox</code>
8	Reply	Classifies the reply's intent and drafts a response.	<code>POST /voice/email-agent/draft-reply</code>

9	Call	For interested prospects: an AI call script, and a test call to your own phone if numbers are set.	GET /voice/agents , POST /voice/call-scripts/generate , POST /sequencer/content/voice/test-call
10	Deal	Creates the deal (one per company per weekend), asks graph8 for the next best step, adds a task.	POST /deals , POST /deals/{id}/next-best-step , POST /tasks

5. Key logic in detail

5.1 How a company is scored (step 2)

Four graph8 lookups run in parallel; only the company lookup is required, the rest are optional and fail quietly.

Signal	Points	Source
Company found in enrichment	+25	/enrichment/lookup/company
GTM profile available	+10	/enrichment/companies/gtm-profile
Weekend engagement signals	+10 each, max 3 (+30)	Step 1
Company appears in ABM intent (last 30 days)	+20	/intent/abm/companies
Hiring for the ICP keywords (default: SDRs, RevOps)	+15	/hiring-signals/companies
Learned adjustment (industry or size you keep approving / rejecting)	+10 / -15	Section 8

A company qualifies when its score is at or above `QUALIFY_THRESHOLD` (default 40). Every point comes with a human-readable reason, which the dashboard shows under *Company · why this score*. Companies that don't qualify keep their score and reasons, so you can see why they were skipped.

Before any paid lookup, the batch drops personal-email domains (gmail, yahoo, outlook, icloud and similar) and non-buyers (`.edu` , `.gov` , `.ac.xx`). The seller's own domain and any Radar competitor are never prospected.

5.2 How the decision-makers are found (step 3)

1. One search: `POST /search/contacts` with the filter `company_domain any_of [domain]` , up to 25 people, cached for 1 hour per domain.
2. Names are cleaned: emoji and symbols removed, casing fixed (for example " 🍌 james" becomes "James").
3. Each person gets a buying role from their title and seniority:
 - **Economic buyer:** chief, CEO, CRO, COO, CFO, VP, president, founder
 - **Champion:** head of, director

- **Influencer:** manager, lead
- **User:** everyone else

4. People are sorted by role and the top 6 kept. Ranking is local, so it costs no extra API calls.

5.3 How emails and contacts are handled

- **Masked emails:** graph8 returns `***` for emails it hasn't revealed. The desk treats anything that isn't a valid address as "no email" (`realEmail()`), and never guesses addresses.
- **Matching contacts back:** after the CRM upsert, contacts are keyed by real email when present *and* by name (`name:first last`), so steps 5, 7, 9 and 10 still find the right graph8 contact id when emails are hidden.
- **Upsert, not insert:** `PUT /contacts/assert/batch` matches on work email, then LinkedIn URL, so running a company twice doesn't duplicate people.
- **Who receives email:** in live mode, *no prospect is ever contacted*. See 5.5.

5.4 How the emails are written (step 5)

For the economic buyer and the next person (usually the champion), the desk calls `POST /sequencer/content/email/generate` with:

- `lead_info` : company name, domain, industry, size, description, every weekend signal, and the scoring reasons;
- `instructions` : open by referencing the topic their team read (without naming readers), focus on ROI for the economic buyer or day-to-day wins for the champion, one proof point, one soft ask for 15 minutes, under 120 words;
- plus, when relevant, a Radar line ("they appear to use X; one respectful line on why teams add or switch; use this battle-card point") and a learned writing rule ("stay under 80 words").

The champion also gets a LinkedIn connection note (`/sequencer/content/linkedin/generate` , under 300 characters). If a generation fails, that message falls back to a local template and is labelled `template` instead of `graph8 AI` , so one failed call never blocks approval.

5.5 Safety: nobody real is emailed

Mode	What step 7 does
Mock	Fake send to fake example domains.
Sandbox	Confirms <code>/sandbox/status</code> , enrolls contacts in a sequence; the sandbox catches every send.
Live (<code>G8_ENV=live</code>)	Dry run: approved messages are recorded as "not sent". With <code>LIVE_SEND_TO</code> set, only that one inbox (yours) is added to the list and enrolled; the backend refuses if it's a prospect's address. Prospects are always marked "held".

Test calls only go to `TEST_CALL_TO` (your own phone). Nothing is sent before a human approves.

5.6 Reply handling (step 8)

An explainable keyword classifier sorts replies into *interested*, *not interested*, *question* or *out of office*. graph8's email agent (`/voice/email-agent/draft-reply`) drafts the response. No real reply arrives in a demo, so a

clearly labelled simulated reply is used; in production this step is fed by graph8's reply webhook.

5.7 Deal and next step (step 10)

The deal is created with every committee contact attached, amount `DEAL_AMOUNT` (default \$25,000), a close date 45 days out, and `DEAL_OWNER_ID` as owner. graph8's next-best-step endpoint suggests the action; if it returns nothing, the desk uses the reply ("Send pricing before Thursday's meeting"). A task is created for it, due in 2 days. Only one deal is created per company per weekend (tracked in the weekend profile store).

6. graph8 Work: Monday brief and approve-by-reply

The sales team already works inside graph8 Work, so the desk posts there. Code: `lib/graph8/work.ts`.

- **Channel:** on first use the backend finds or creates `#autopilot-revenue-desk` (`GET/POST /work/channels`). Change it with `WORK_CHANNEL`, or set it empty to turn Work off.
- **Monday brief:** when every company in a weekend run is finished or waiting for approval, the backend posts the funnel and the ranked company list (`POST /work/messages`).
- **Approval requests:** each approval gets a 4-character code (for example `4F2A`) and is posted with the score, top reasons, competitor (if any) and email subjects.
- **Approve by reply:** every 5 seconds the backend reads the channel (`GET /work/conversations/{id}/messages`). A message that is exactly `approve 4F2A` or `reject 4F2A` (also `approved / yes / rejected / no`), posted after the request, decides it. The dashboard then shows “via graph8 Work”. It works from the Work mobile app too.
- graph8's `/agent/approvals` inbox only holds approvals raised by graph8's own agents (the API has no create endpoint), which is why the desk uses Work for approvals inside graph8.
- Every Work call is best-effort: if Work is down, the dashboard still works.

7. Radar competitor detection

Code: `lib/pipeline/competitors.ts`. Runs inside step 2.

1. Reads the seller's tracked competitors: `GET /radar/competitors` (cached 10 minutes).
2. If the prospect is the seller (`SELLER_DOMAIN`) or a tracked competitor, it is marked not qualified: “Skipped: this is X (tracked in Radar)”.
3. Otherwise it searches what we know about the prospect (description, GTM profile, signals) for each competitor's name or domain as a whole word.
4. On a match it fetches `GET /radar/competitors/{id}/battle-card` and keeps up to 3 short talking points (differentiators, win themes, objection handling).
5. The score card shows an orange “Uses competitor” box with the evidence and points, the Work approval mentions it, and the email gets one respectful “why teams add or switch” line.

Radar only knows the competitors you track. Add them in graph8 → Radar (for example Apollo, ZoomInfo, Clay, HubSpot) before the demo.

8. Learning from decisions

Code: `lib/pipeline/learning.ts`. Every decision is saved with the company's industry, size band (under 50 / 50–199 / 200–999 / 1000+), score, whether drafts were edited, and how much shorter they became.

Lesson	Rule	Effect
Industry rejected	≥ 2 rejections and more rejections than approvals	–15 points for that industry

Industry approved	≥ 2 approvals and more approvals than rejections	+10 points
Size band rejected / approved	Same rule on size band	-15 / +10
Drafts shortened	≥ 2 approvals where the email lost 20%+ of its words	AI told to stay under 80 words

Two matching decisions are needed, so one click never swings the model. Lessons appear in the *Learning from you* card and, for affected companies, in a purple “Learned from your decisions” box on the score card.

9. Weekend batch and demo weekend

`POST /api/batch` (the **Run weekend** button) runs every engaged company in parallel, up to 10. The shared rate limiter keeps the whole batch under graph8's limit.

- **Mock mode:** five fictional companies on `.example` domains plus 31 personal-email clicks: three good fits, one tiny shop and one university that are filtered out.
- **Live mode:** reads real visitor events. A new graph8 org has no click history, so when none are found (or `{"demo": true}` is sent) it uses the **demo weekend**: 9 clicks on real software companies (PostHog, Supabase, Retool, Airbyte, Linear), three personal emails and one university. Everything after the click runs on real graph8 data. The dashboard labels this `demo clicks · real graph8 data`.
- **Real intake:** newsletter clicks and site visits can also arrive by webhook (`/api/webhooks/content-engagement` , `/api/webhooks/website-visit`). They are aggregated into one profile per company per weekend (Friday to Monday), and one run starts per company.

10. Dashboard (Mission Control) and app tour

Area	Shows
Header	Mode badge (MOCK MODE, SANDBOX, LIVE · SENDS HELD, BACKEND OFFLINE), Tour, domain box, Run one, Run weekend, Reset demo
This weekend	Funnel (clicks → companies → good fits → decision-makers → emails ready → sent → deals), skipped counts, Monday brief status, table of companies with Review buttons
Pipeline	The 10 steps with live status, skip reasons and "via graph8 Work"
Learning from you	Approved / rejected / edited counts and current lessons
Approval box	Summary, the graph8 Work code, Approve and Reject
Company · why this score	Score, each reason, learned adjustments, Radar competitor, what their team engaged with
Buying committee	People with title and role
AI-drafted messages	Each draft, labelled graph8 AI or template; editable while waiting for approval
Outbox, Reply + call, Deal + next step, Event log	Results of steps 7 to 10 and the raw event stream

App tour: a 9-step guided walkthrough that dims the page and highlights one area at a time. It opens on the first visit (remembered in the browser) and the **Tour** button reopens it. Keyboard: arrows to move, Esc to close.

11. Modes and configuration

Mode	Settings	Use it for
Mock	<code>USE MOCKS=true</code>	Practice and backup demo; no graph8 calls, no key needed
Live	<code>USE MOCKS=false</code> , <code>G8_ENV=live</code>	Real graph8 data on the hackathon org; sends held
Sandbox	<code>USE MOCKS=false</code> (default <code>G8_ENV</code>)	When graph8's developer sandbox is available

Backend environment variables (`backend/.env`)

Variable	Default	Meaning
<code>G8_API_KEY</code>	(none)	graph8 API key (Settings → Developer → API Keys)
<code>G8_BASE_URL</code>	<code>https://be.graph8.com/api/v1</code>	API base URL
<code>USE MOCKS</code> / <code>G8_ENV</code>	true / sandbox	Mode, see above
<code>LIVE_SEND_TO</code>	(empty)	Your own inbox; the only address enrolled in live mode

PORT / CORS_ORIGIN	4000 / *	Server port; allowed frontend origin
QUALIFY_THRESHOLD	40	Minimum score to continue
ICP_HIRING_KEYWORDS	SDR, revenue operations	Roles that count as a hiring signal
G8_MAX_RPS	4	Requests per second to graph8
COMMITTEE_USE_ORG_CHART	false	Paid org-chart research (costs credits)
AGENT_NAME / SENDER_NAME	default-agent / Alex	AI writer persona and email signature
SEQUENCE_ID / SENDER_EMAIL	(empty)	Sequence to enrol into, or mailbox to create one
DEAL_OWNER_ID / DEAL_AMOUNT	(empty) / 25000	Deal owner (your graph8 user id) and value
VOICE_AGENT_ID	(first agent)	Voice agent for call scripts and test calls
TEST_CALL_FROM / TEST_CALL_TO	(empty)	Agent's graph8 number / your own phone
WORK_CHANNEL	autopilot-revenue-desk	graph8 Work channel; empty turns Work off
SELLER_DOMAIN	graph8.com	Your company; never prospected
WEBHOOK_SECRET	(empty)	Secret for verifying graph8 webhook signatures

Frontend: NEXT_PUBLIC_API_URL in frontend/.env.local (default http://localhost:4000).

12. HTTP API reference

Base URL `http://localhost:4000` . All bodies are JSON. CORS is enabled for the frontend origin.

Method and path	Body	Returns
GET /api/health	(none)	<code>{ ok, mode, weekendKey, product }</code>
POST /api/run	<code>{ domain?, event? }</code>	202 <code>{ runId }</code>
POST /api/batch	<code>{ max?, demo? }</code>	202 <code>{ batchId, totalClicks, freemail, nonBuyer, domains, runIds, demo }</code>
GET /api/runs?batchId=	(none)	<code>{ batch, runs[], brief }</code> , one row per company
GET /api/events?runId=	(none)	Server-Sent Events stream of pipeline events
POST /api/approvals/{approvalId}	<code>{ decision: "approve" "reject", drafts? }</code>	<code>{ ok, decision }</code> ; 404 if not pending
GET /api/learning	(none)	<code>{ decisions, approved, rejected, edited, lessons[] }</code>
POST /api/learning/reset	(none)	Cleared summary
GET /api/outbox	(none)	Sandbox outbox (empty in mock and live)
POST /api/demo/reset	(none)	Clears the event log (and restores the sandbox snapshot in sandbox mode)
POST /api/webhooks/content-engagement	<code>{ email, rawUrlClicked, title?, timestamp? }</code>	Records a newsletter click; starts a run
POST /api/webhooks/website-visit	<code>{ email domain, pageUrl, title?, timestamp?, visitorId? }</code>	Records a site visit; starts a run
GET /api/pipeline?weekendKey=	(none)	Monday inbox of weekend company profiles
GET /api/pipeline/{domain}	(none)	One profile plus its run events
POST /api/pipeline/{domain}/approve	<code>{ decision }</code>	Decides that company's pending approval
POST /api/webhooks/graph8	graph8 signed delivery	Verifies <code>X-Studio-Signature</code> (HMAC-SHA256 of <code>timestamp.body</code>), dedupes by id

Pipeline event shape

```
{ "id": 42, "runId": "run_3fa1c2d9", "step": "qualify",  
  "status": "done", // pending | running | done | failed | skipped | awaiting_approval  
  "data": { "company": {...}, "score": 76, "qualified": true, "reasons": [...], "competitor": {...},  
  "learned": [...] },  
  "timestamp": "2026-09-27T09:12:03.120Z" }
```

13. Postman: collection and sample requests

Import `docs/Autopilot-Revenue-Desk.postman_collection.json` into Postman. It has a `baseUrl` variable (default `http://localhost:4000`) and saves `runId`, `batchId` and `approvalId` automatically between requests. Sample data uses reserved `.example` domains.

Health

```
GET {{baseUrl}}/api/health
```

```
200 { "ok": true, "mode": "live", "weekendKey": "2026-W-09-25", "product": "Autopilot Revenue Desk"  
}
```

Run one company, with a newsletter click

```
POST {{baseUrl}}/api/run
```

```
Content-Type: application/json
```

```
{ "event": { "type": "newsletter_click", "domain": "acme-software.example",  
  "title": "Read \"Scaling your API to 10k requests/sec\" (3 readers)",  
  "url": "https://example.com/blog/api-scaling" } }
```

```
202 { "runId": "run_3fa1c2d9" }
```

Run the weekend

```
POST {{baseUrl}}/api/batch
```

```
{ "max": 10 }
```

```
202 { "batchId": "batch_94de4b1b", "totalClicks": 40, "freemail": 31, "nonBuyer": 0,  
  "domains": ["northwind-analytics.example", "acme-software.example", "..."],  
  "runIds": ["run_...", "run_..."] }
```

Monday summary

```
GET {{baseUrl}}/api/runs?batchId={{batchId}}

200 { "runs": [ { "runId": "run_...", "company": "Acme Software", "score": 76, "qualified": true,
                  "people": 4, "drafts": 2, "sent": 0, "status": "awaiting_approval", "approvalId":
"appr_1ebedcdf" } ],
      "brief": { "status": "done", "kind": "brief", "channel": "autopilot-revenue-desk", "text":
"🌞 Monday brief ..." } } }
```

Approve, with an edited draft

```
POST {{baseUrl}}/api/approvals/{{approvalId}}
{ "decision": "approve",
  "drafts": [ { "contactEmail": "dana.whitfield@acme-software.example", "channel": "email",
                "subject": "Acme and API scaling",
                "body": "Hi Dana,\n\nYour team has been reading about scaling APIs
...\n\nBest,\nAlex" } ] }

200 { "ok": true, "decision": "approve" }
```

Reject

```
POST {{baseUrl}}/api/approvals/{{approvalId}}
{ "decision": "reject" }
```

Newsletter click webhook

```
POST {{baseUrl}}/api/webhooks/content-engagement
{ "email": "reader@acme-software.example",
  "rawUrlClicked": "https://example.com/blog/api-scaling?utm_source=newsletter",
  "title": "Scaling your API to 10k requests/sec", "timestamp": "2026-09-26T10:15:00Z" }

200 { "ok": true, "message": "Newsletter click recorded; pipeline started", "runId": "run_...",
      "startedRun": true, "profile": {...} }
```

Website visit webhook

```
POST {{baseUrl}}/api/webhooks/website-visit
{ "domain": "acme-software.example", "pageUrl": "https://example.com/pricing",
  "title": "Pricing", "timestamp": "2026-09-27T08:30:00Z", "visitorId": "v_123" }
```

Learning

```
GET {{baseUrl}}/api/learning

200 { "decisions": 6, "approved": 4, "rejected": 2, "edited": 4,
      "lessons": [ "You rejected 2 Healthtech SaaS companies: they now score 15 lower",
                    "You approved 4 companies with 200-999 employees: they now score 10 higher" ] }
```

Listen to live events (terminal)

```
curl -N "http://localhost:4000/api/events?runId=run_3fa1c2d9"
```

14. How to run and test every feature

Setup

```
# backend
cd backend
npm install
copy .env.example .env      # then fill in G8_API_KEY etc., or keep USE MOCKS=true
npm start                   # http://localhost:4000

# frontend (second terminal)
cd frontend
npm install
npm run build
npm start                   # http://localhost:3000
```

Automated checks: `npm run typecheck` in both folders, `npm run lint` in the frontend. GitHub Actions runs the backend typecheck and mock smoke test on every push.

Feature test checklist

Feature	How to test	Expected result
Backend up / mode	<code>GET /api/health</code> , or look at the header badge	<code>mode</code> is mock, live or sandbox
Full pipeline (one company)	Type a domain, click Run one (or <code>POST /api/run</code>)	Steps turn green; stops at <i>awaiting approval</i>
Weekend batch + funnel	Click Run weekend	Funnel fills; personal-email and student clicks counted as skipped; low-fit companies “filtered” with scores
Why this score	Open any company	Every point listed as a reason
Decision-makers	Open a qualified company	Up to 6 people with roles; names clean
CRM save (live)	After step 4, open graph8 → CRM → Companies / Contacts	Company, list and contacts exist
AI emails	After step 5	Drafts labelled <i>graph8 AI</i> , mentioning the article
Edit before approve	Change a draft's text, click Approve	Outbox and learning use the edited text
Reject	Click Reject	Later steps “skipped: Rejected by human”; nothing sent
Approve in graph8 Work (live)	In graph8 Work open <code>#autopilot-revenue-desk</code> , reply <code>approve CODE</code> using the code shown in the approval box	Within about 5 s the run continues; Pipeline shows “via graph8 Work”

Monday brief (live)	Run weekend and wait for all companies to settle	Brief posted in the Work channel; panel shows "posted ✓"
Send safety (live)	Approve a company	Outbox shows "dry run: not sent" or "held"; no prospect emailed
Reply + call	After send	Reply classified <i>interested</i> , response drafted; call script (or test call to your phone)
Deal (live)	After step 10, open graph8 → CRM → Deals	"Company · Weekend inbound", \$25,000, next step task
Radar competitor	Mock: run weekend, open Acme Software. Live: add a competitor in Radar whose name appears in a prospect's profile	Orange "Uses competitor" box with talking points; email mentions it
Never prospect a competitor	<code>POST /api/run {"domain":"graph8.com"}</code>	Stops at qualify: "Competitor: your own company"
Learning	Mock: run weekend twice, approve Northwind and Acme, reject Fabrikam each time	Learning card shows lessons; second run's scores change with "Learned from your decisions"
Webhook intake	Send the content-engagement sample in Postman	Run starts; <code>GET /api/pipeline</code> lists the company profile
App tour	Click Tour (or open the page in a private window)	9 steps highlighting each area
Reset	Click Reset demo	Dashboard cleared, pending approvals rejected
Backend down	Stop the backend, reload the page	Badge shows BACKEND OFFLINE; Run disabled

15. graph8 endpoints used

Area	Endpoints
Signals	<code>GET /visitors/companies/events</code> , <code>GET /visitors/links/events</code> , <code>GET /newsletters/{id}/issues/{id}/analytics/subscribers/{id}/timeline</code> , <code>GET /intent/abm/companies</code> , <code>POST /hiring-signals/companies</code>
Enrichment and search	<code>POST /enrichment/lookup/company</code> , <code>GET /enrichment/companies/gtm-profile</code> , <code>POST /search/contacts</code> , <code>POST /voice/research/org-chart</code> (optional)
CRM	<code>POST /lists</code> , <code>POST /companies</code> , <code>PUT /contacts/assert/batch</code> , <code>GET /contacts</code> , <code>POST /deals</code> , <code>POST /deals/{id}/next-best-step</code> , <code>POST /tasks</code>
AI content	<code>POST /sequencer/content/email/generate</code> , <code>POST /sequencer/content/linkedin/generate</code> , <code>POST /voice/email-agent/draft-reply</code> , <code>POST /voice/call-scripts/generate</code>
Sequences	<code>GET/POST /sequences</code> , <code>POST /sequences/{id}/contacts</code>
Voice	<code>GET /voice/agents</code> , <code>POST /sequencer/content/voice/test-call</code>

Radar	GET /radar/competitors , GET /radar/competitors/{id}/battle-card
Work	GET/POST /work/channels , POST /work/messages , GET /work/conversations/{id}/messages
Sandbox	GET /sandbox/status , GET /sandbox/outbox , POST /sandbox/fixtures/seed snapshot restore
Webhooks	Signed deliveries verified at POST /api/webhooks/graph8

16. Known limits and next steps

- **Real sending:** new mailboxes need about two weeks of warm-up before sequences send, so sends are held on the live system. With a warmed mailbox, set `SEQUENCE_ID` and step 7 enrolls contacts for real.
- **Click history:** the hackathon org has no visitor data yet, so the demo weekend stands in for step 1. Connecting graph8 site tracking or the newsletter webhooks replaces it.
- **Voice test call:** needs a phone number assigned to the voice agent in graph8.
- **Replies:** a simulated reply is used in demos; production would feed step 8 from graph8's reply webhook.
- **Reset in live mode** clears the dashboard but not the records created in graph8.
- **Scale:** events live in memory in one process; a production version would move them to a queue and a database (for example Postgres) and run several workers.